

Sistema de Control Difuso Mamdani para Robot Seguidor de Línea basado en STM32F411

Kevin Adrian Lara Hernandez, Orlando Contreras Reyes, Roberto Lopez Rodriguez¹

Departamento de Ingeniería Electrónica, Universidad Autónoma de Aguascalientes,
Aguascalientes, México

464376@edu.uaa.mx, 348390@edu.uaa.mx, 234512@edu.uaa.mx

Resumen Este trabajo presenta el diseño e implementación de un sistema de control difuso tipo Mamdani para un robot seguidor de línea. El sistema emplea dos sensores reflectivos (ADC de 12 bits) como entradas y genera comandos de velocidad para dos motores de DC mediante modulación por ancho de pulso (PWM). La base de reglas cubre nueve combinaciones de estados sensoriales (negro, gris, blanco), la inferencia se realiza con la operación mínimo-máximo y la defuzzificación mediante el método del centroide integrado numéricamente. El firmware fue desarrollado en C sobre el microcontrolador STM32F411CEU6 usando el HAL de STMicroelectronics, con DMA para la adquisición simultánea de los dos canales del ADC.

Keywords: Lógica difusa · Mamdani · Centroide · Seguidor de línea · STM32F411 · PWM · DMA

1. Introducción

Los robots seguidores de línea son una plataforma clásica para evaluar estrategias de control en tiempo real. La lógica difusa resulta especialmente apropiada en este contexto porque los sensores ópticos no producen lecturas binarias perfectas; en las transiciones entre negro y blanco existe una región de incertidumbre que los conjuntos difusos modelan de forma natural [1].

El controlador Mamdani [2] permite expresar el conocimiento del experto como reglas lingüísticas del tipo «*Si el sensor izquierdo está en negro y el derecho en blanco, entonces el motor izquierdo va rápido y el derecho va lento*», lo que simplifica el ajuste del sistema sin necesidad de un modelo matemático explícito del robot.

En este reporte se describe la arquitectura completa del firmware: funciones de pertenencia, base de reglas, motor de inferencia y defuzzificador analítico/-numérico, así como la configuración de los periféricos del STM32F411 (TIM1, ADC1 con DMA y GPIO).

2. Plataforma de hardware

2.1. Microcontrolador

El sistema se implementa sobre un módulo STM32F411CEU6 (*Black-Pill*), que opera a 100 MHz gracias a un cristal externo de 25 MHz con multiplicador PLL ($M=12$, $N=96$, $P=2$). Sus características más relevantes para esta aplicación son: ADC de 12 bits con modo de escaneo y trigger externo, cuatro canales de DMA2, y el temporizador avanzado TIM1 con salidas complementarias y generación de trigger para el ADC.

2.2. Sensores

Se emplean dos sensores reflectivos de infrarrojo conectados a PA0 (sensor derecho) y PA1 (sensor izquierdo). Las lecturas quedan almacenadas en el arreglo `carro.Sensor[]` mediante DMA circular, disparado por el canal 3 de TIM1 (modo *toggle*) a la misma frecuencia que el PWM de los motores (20 kHz), garantizando una muestra por ciclo PWM.

2.3. Actuadores

Dos motores de DC son controlados por un puente H. Las señales de dirección (AIN1/AIN2 para el motor derecho, BIN1/BIN2 para el izquierdo) se fijan al inicio en modo avance, mientras que la velocidad se regula con los canales CH1 y CH2 de TIM1 en modo PWM. El periodo del contador es de 4999 cuentas, lo que corresponde a $f_{PWM} = 100 \text{ MHz} / 5000 = 20 \text{ kHz}$.

2.4. Fotografía del robot

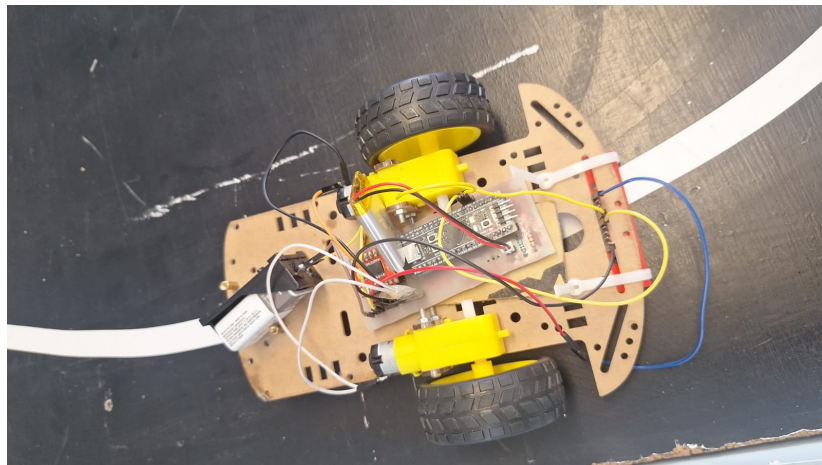


Figura 1: Prototipo de robot seguidor de línea basado en STM32F411CEU6.

3. Diseño del sistema de lógica difusa

El sistema difuso tiene la siguiente estructura:

- **Entradas:** lectura ADC del sensor izquierdo (`left_raw`) y del sensor derecho (`right_raw`).
- **Salidas:** porcentaje de velocidad para el motor izquierdo (MI) y motor derecho (MD), en el rango $[0, 100]$.
- **Método de inferencia:** Mamdani con operadores mínimo (AND) y máximo (OR).
- **Defuzzificación:** centroide (centro de gravedad).

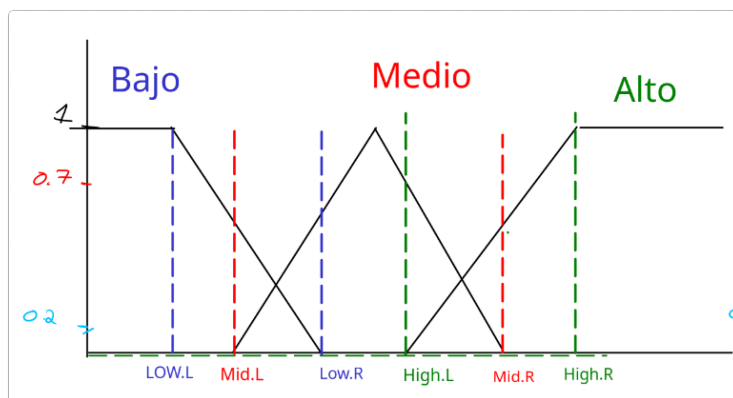


Figura 2: Funciones de pertenencia del motores

3.1. Conjuntos difusos de entrada (sensores)

Cada sensor se clasifica en tres conjuntos lingüísticos definidos sobre el rango de salida del ADC (0–4095 cuentas):

Cuadro 1: Parámetros de los conjuntos difusos de entrada.

Conjunto	L	C	R	Tipo
Blanco (<i>White</i>)	3 500	—	3 600	Trapezoide izq.
Gris (<i>Gray</i>)	3 600	3 760	3 900	Triangular
Negro (<i>Black</i>)	3 900	—	4 020	Trapezoide der.

Las funciones de pertenencia son:

$$\mu_{\text{White}}(x) = \begin{cases} 1 & x \leq 3500 \\ \frac{3600 - x}{3600 - 3500} & 3500 < x < 3600 \\ 0 & x \geq 3600 \end{cases} \quad (1)$$

$$\mu_{\text{Gray}}(x) = \begin{cases} 0 & x \leq 3600 \\ \frac{x - 3600}{3760 - 3600} & 3600 < x \leq 3760 \\ \frac{3900 - x}{3900 - 3760} & 3760 < x < 3900 \\ 0 & x \geq 3900 \end{cases} \quad (2)$$

$$\mu_{\text{Black}}(x) = \begin{cases} 0 & x \leq 3900 \\ \frac{x - 3900}{4020 - 3900} & 3900 < x < 4020 \\ 1 & x \geq 4020 \end{cases} \quad (3)$$

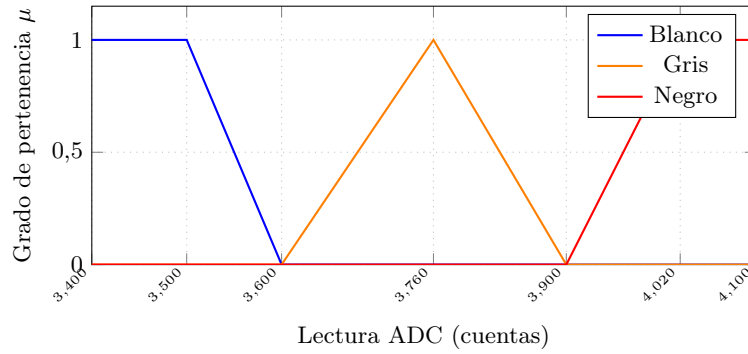


Figura 3: Funciones de pertenencia de los sensores de entrada.

3.2. Conjuntos difusos de salida (motores)

Las salidas MI y MD comparten el mismo universo de discurso $[0, 100]$ (% de velocidad) con tres conjuntos:

Cuadro 2: Parámetros de los conjuntos difusos de salida.

Conjunto	L	C	R	Tipo
Bajo (<i>Low</i>)	0	—	50	Trapezoide izq.
Medio (<i>Mid</i>)	25	50	75	Triangular
Alto (<i>High</i>)	50	—	100	Trapezoide der.

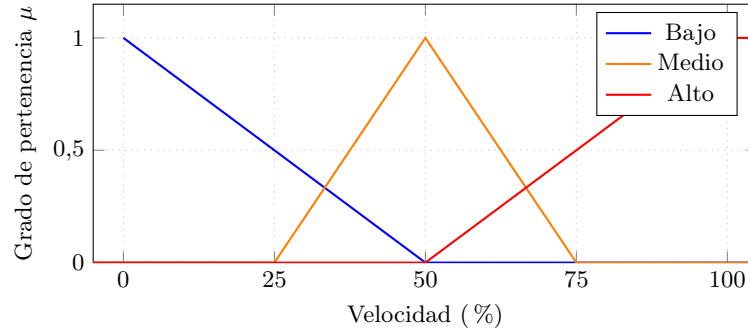


Figura 4: Funciones de pertenencia de las salidas (motores MI y MD).

4. Base de reglas Mamdani

La Tabla 3 muestra las nueve reglas que componen la base de conocimiento del controlador. Cada regla tiene la forma:

$$\text{SI } L \in A_L \text{ AND } R \in A_R \Rightarrow \text{MI} \in B_{\text{MI}}, \text{MD} \in B_{\text{MD}}$$

donde L y R son las lecturas del sensor izquierdo y derecho respectivamente, A_* son conjuntos de entrada y B_* conjuntos de salida. El parámetro *peso* escala la fuerza de la regla antes de la agregación.

Cuadro 3: Base de reglas Mamdani (L = sensor izq., R = sensor der.).

#	Sensor L	Sensor R	MI (izq.)	MD (der.)	Peso
1	Blanco	Blanco	Alto	Alto	1.0
2	Gris	Blanco	Alto	Medio	2.0
3	Blanco	Gris	Medio	Alto	2.0
4	Negro	Blanco	Alto	Bajo	2.0
5	Blanco	Negro	Bajo	Alto	2.0
6	Gris	Gris	Medio	Medio	1.0
7	Negro	Gris	Alto	Bajo	1.0
8	Gris	Negro	Bajo	Alto	1.0
9	Negro	Negro	Bajo	Alto	1.0

5. Motor de inferencia

5.1. Fuzzificación

Para cada sensor se calculan los tres grados de pertenencia mediante las funciones definidas en las ecuaciones (1)–(3). La implementación en `FuzzyLogic.c` utiliza las funciones genéricas `Trapz()` y `Triang()`:

```

1 float Trapz(float x, FuzzyArgs args, uint8_t type)
2 {
3     if(type == TRAPZ_LOW){
4         if(x <= args.L) return 1.0f;
5         if(x < args.R) return (args.R - x)/(args.R - args.L)
6     ;
7         return 0.0f;
8     }
9     if(type == TRAPZ_HIGH){
10        if(x <= args.L) return 0.0f;
11        if(x < args.R) return (x - args.L)/(args.R - args.L)
12    ;
13        return 1.0f;
14    }
15 }
16
17 float Triang(float x, FuzzyArgs args) {
18     if(x <= args.L) return 0.0f;
19     if(x < args.C && x > args.L) return (x-args.L)/(args.C-
20     args.L);
21     if(x < args.R && x > args.C) return (args.R-x)/(args.R-
22     args.C);
23     return 0.0f;
24 }

```

Listing 1.1: Funciones de pertenencia genéricas (FuzzyLogic.c).

5.2. Evaluación de reglas

La función `Mamdani_Infer()` recorre la base de reglas, calcula la fuerza de disparo de cada regla como $\alpha_i = w_i \cdot \min(\mu_L, \mu_R)$ y la agrega por motor mediante el operador máximo:

$$y_{\text{LOW}} = \max_i \{\alpha_i \mid \text{consecuente}(i) = \text{LOW}\} \quad (4)$$

y análogamente para los conjuntos MEDIO y ALTO.

5.3. Defuzzificación por centroide numérico

La función `Centroide()` implementa la integral discreta:

$$u^* = \frac{\sum_{x=0}^{100} x \cdot \mu_{\text{agg}}(x)}{\sum_{x=0}^{100} \mu_{\text{agg}}(x)} \quad (5)$$

donde $\mu_{\text{agg}}(x) = \max(\hat{\mu}_{\text{LOW}}, \hat{\mu}_{\text{MID}}, \hat{\mu}_{\text{HIGH}})$ con $\hat{\mu}_k(x) = \min(y_k, \mu_k(x))$. El paso de integración es $\Delta x = 0,1$, dando 1001 puntos en el universo de discurso.

```

1 float Centroide(float y_low, float y_mid, float y_high)
2 {
3     float num = 0.0f, den = 0.0f;
4     for(float x = 0.0f; x <= 100.0f; x += 0.1f){
5         float low  = fminf(y_low,
6                             FuzzyMembership(x, Low,  TRAPZ_LOW
7     ));
8         float mid  = fminf(y_mid,
9                             FuzzyMembership(x, Mid,  TRIANG));
10        float high = fminf(y_high,
11                             FuzzyMembership(x, High,
12     TRAPZ_HIGH));
13        float mu    = fmaxf(low, fmaxf(mid, high));
14        num += x * mu;
15        den += mu;
16    }
17    if(den == 0.0f) return 0.0f;
18    return num / den;

```

Listing 1.2: Defuzzificación por centroide numérico (Centroide.c).

6. Cálculo analítico de áreas (módulo Centroide)

Adicionalmente al centroide numérico, el módulo `Centroide.c` implementa funciones analíticas de área y centroide para cada región de la función de salida agregada. Esto fue diseñado como alternativa de mayor eficiencia computacional.

6.1. Inicialización (Centroide_Init)

Durante la inicialización se precomputan las intersecciones entre conjuntos adyacentes:

$$Y_1 = \frac{R_{\text{Low}} \cdot C_{\text{Mid}} - L_{\text{Low}} \cdot L_{\text{Mid}}}{(C_{\text{Mid}} - L_{\text{Mid}}) + (R_{\text{Low}} - L_{\text{Low}})} \quad (6)$$

$$Y_2 = \frac{R_{\text{High}} \cdot R_{\text{Mid}} - L_{\text{High}} \cdot C_{\text{Mid}}}{(R_{\text{Mid}} - C_{\text{Mid}}) + (R_{\text{High}} - L_{\text{High}})} \quad (7)$$

6.2. Regiones de área

Se definen cinco funciones de área: `Area_Low`, `Area_Mid`, `Area_High`, `Area_InterLm` (intersección Low–Mid) y `Area_InterMh` (intersección Mid–High). Cada una devuelve una estructura `DefuzzArgs` con numerador ($\sum A_i C_i$) y denominador ($\sum A_i$) para aplicar el teorema del centroide compuesto:

$$u^* = \frac{\sum_i A_i \cdot C_i}{\sum_i A_i} \quad (8)$$

7. Configuración de periféricos STM32F411

7.1. Reloj del sistema

Cuadro 4: Configuración de la PLL del STM32F411.

Parámetro Valor	
Fuente	HSE 25 MHz
PLLM	12
PLLN	96
PLL P	2
SYSCLK	100 MHz
HCLK	100 MHz
APB1	50 MHz
APB2	100 MHz

7.2. TIM1 – PWM y trigger ADC

TIM1 opera en modo *up-counting* con prescaler 0 y periodo $ARR = 4999$, lo que resulta en una frecuencia de $f = 100 \text{ MHz} / 5000 = 20 \text{ kHz}$.

- **CH1 (PA8):** PWM modo 1, motor derecho.
- **CH2 (PA9):** PWM modo 1, motor izquierdo.
- **CH3 (interno):** modo *toggle*, genera TIM1_TRGO para disparar el ADC por hardware (ADC_EXTERNALTRIGCONV_T1_CC3).

El ciclo de trabajo se actualiza en el lazo principal:

```

1 __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1,
2   (uint32_t)(cmd.left * 0.5f * PWM_PERIOD / 100.0f));
3 __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2,
4   (uint32_t)(cmd.right * 0.5f * PWM_PERIOD / 100.0f));

```

Listing 1.3: Actualización de PWM en el lazo principal (`main.c`).

El factor 0.5 limita el ciclo de trabajo máximo al 50 % del periodo, evitando sobrecalentamiento del puente H en condiciones de velocidad plena.

7.3. ADC1 con DMA

El ADC1 opera en modo de escaneo de dos canales (CH0 → sensor derecho, CH1 → sensor izquierdo), con trigger externo ascendente proveniente de TIM1_CH3. El DMA2_Stream0 transfiere automáticamente los resultados al arreglo `carro.Sensor[2]` con modo circular, garantizando que el lazo principal siempre lea la muestra más reciente sin overhead de CPU.

7.4. GPIO

Las señales de dirección del puente H se configuran como salidas push-pull en el puerto GPIOA:

Cuadro 5: Asignación de pines de dirección.

Pin	Señal	Función
PA2	AIN1	Dirección motor derecho (1)
PA3	AIN2	Dirección motor derecho (2)
PA4	BIN1	Dirección motor izquierdo (1)
PA5	BIN2	Dirección motor izquierdo (2)

Al inicio, AIN1 y BIN1 se ponen en nivel alto (avance).

8. Arquitectura del software

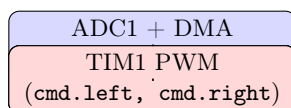


Figura 5: Diagrama de flujo del sistema de control difuso.

La estructura modular del proyecto se compone de cuatro archivos fuente independientes:

FuzzyLogic.c/.h Funciones primitivas de pertenencia (**Trapz**, **Triang**), operadores difusos (AND, OR) y funciones de pertenencia específicas para sensores y motores.

Mamdani.c/.h Base de reglas, motor de inferencia (**Mamdani_Infer**) y función de alto nivel (**Mamdani_MotorCommand**).

Centroide.c/.h Defuzzificador numérico (**Centroide**) y funciones analíticas de área (**Area_Low**, **Area_Mid**, **Area_High**, **Area_InterLm**, **Area_InterMh**).

main.c/.h Configuración HAL, lazo principal y estructura **CarroData**.

Cuadro 6: Desempeño cualitativo por escenario de prueba.

Escenario	Seguimiento	Oscilación
Línea recta	—	—
Curva suave	—	—
Curva cerrada	—	—
Pérdida de línea	—	—

9. Discusión

La base de reglas de nueve entradas cubre de forma exhaustiva el espacio de estados sensoriales binario-difuso. Los pesos de 2.0 asignados a las reglas de corrección de rumbo (filas 2–5 de la Tabla 3) aumentan la influencia de esas reglas sobre el motor de agregación sin alterar la estructura Mamdani.

El ciclo de defuzzificación numérica con $\Delta x = 0,1$ implica 1001 iteraciones por motor, totalizando 2002 evaluaciones de `FuzzyMembership` por ciclo de control. A 96 MHz esta carga es asumible, pero las funciones analíticas de `Centroide.c` podrían reducirla significativamente si el tiempo de cómputo se convierte en un cuello de botella.

10. Conclusiones

Se diseñó e implementó exitosamente un controlador difuso tipo Mamdani para un robot seguidor de línea sobre el STM32F411CEU6. El sistema integra adquisición analógica con DMA, generación de PWM sincronizada y una cadena completa de inferencia difusa organizada en módulos independientes y reutilizables.

Como trabajo futuro se propone:

1. Validar las funciones analíticas de centroide y sustituir el método numérico para reducir el tiempo de cómputo.
2. Ajustar los umbrales de los conjuntos difusos de entrada mediante calibración en línea.
3. Agregar una regla de frenado controlado cuando ambos sensores detecten negro por más de N ciclos consecutivos.

Referencias

1. Zadeh, L.A.: Fuzzy sets. *Information and Control* **8**(3), 338–353 (1965). [https://doi.org/10.1016/S0019-9958\(65\)90241-X](https://doi.org/10.1016/S0019-9958(65)90241-X)
2. Mamdani, E.H., Assilian, S.: An experiment in linguistic synthesis with a fuzzy logic controller. *International Journal of Man-Machine Studies* **7**(1), 1–13 (1975). [https://doi.org/10.1016/S0020-7373\(75\)80002-2](https://doi.org/10.1016/S0020-7373(75)80002-2)

3. Ross, T.J.: Fuzzy Logic with Engineering Applications, 3rd edn. Wiley, Chichester (2010).
4. STMicroelectronics: STM32F411xC/E Reference Manual (RM0383). Rev. 3 (2023). https://www.st.com/resource/en/reference_manual/rm0383-stm32f411xce-advanced-armbased-32bit-mcus-stmicroelectronics.pdf